

Guia de Estudo — Como Explicar Nossa Arquitetura Sem Jargão

Material de apoio para apresentação — explicando cada decisão técnica com analogias do dia a dia
Gerado em 03/07/2026

Como usar este guia: cada tópico tem 3 partes — **O que é** (a explicação técnica curta), **Analogia** (a forma de explicar para quem não é da área) e **Por que escolhemos** (a frase de defesa, caso alguém pergunte "por que não fazer diferente?"). No final tem um roteiro sugerido de apresentação com tempo estimado por bloco.

Parte 0 — A ideia central, antes de entrar em detalhe

Antes de explicar qualquer tecnologia específica, a plateia precisa entender **um** conceito — todo o resto é consequência dele:

Separamos "o que o sistema faz" de "com o que o sistema é feito".

Analogia da cozinha de um restaurante: a *receita* de um prato (quanto sal, quanto tempo no forno, em que ordem) não muda dependendo de qual fogão o restaurante compra. Se trocar o fogão a gás por um de indução, a receita continua a mesma — só muda quem manuseia o fogão. No nosso sistema, a "receita" é a regra de negócio (ex.: "um pedido só pode ser cancelado se ainda não foi entregue"); o "fogão" é o banco de dados, o cache, o framework. Construímos o sistema para que trocar o fogão nunca exija reescrever a receita.

Essa frase única — **regra de negócio separada de detalhe técnico** — é o fio condutor de tudo que vem a seguir. Se a plateia entender só isso, já entendeu 80% do valor da arquitetura.

Parte 1 — Backend (a API)

1.1 As três camadas (Clean Architecture)

O que é: o código é organizado em três círculos concêntricos. O círculo de fora (**infra**) conhece os de dentro, mas nunca o contrário.

Analogia do prédio: pense em um prédio com fundação, estrutura e acabamento.

- A **fundação** (`core`) é genérica — vigas e concreto que servem para qualquer prédio, não importa se vai virar hospital ou escritório.
- A **estrutura** (`domain`) é o projeto do arquiteto — define quantos quartos, onde fica a cozinha, as regras de uso daquele prédio específico. O arquiteto não precisa saber qual marca de tijolo o pedreiro vai usar.
- O **acabamento** (`infra`) é o pedreiro escolhendo o material de verdade — essa marca de tijolo, aquele fornecedor de tinta. Pode trocar de fornecedor sem alterar o projeto do arquiteto.

Por que escolhemos: se amanhã a empresa decidir trocar de banco de dados, ou de provedor de nuvem, só mexemos no "acabamento" — a regra de negócio (o "projeto do arquiteto") não é tocada. Isso também significa que testamos a regra de negócio sem precisar "construir o prédio inteiro" (subir banco de dados de verdade) toda vez.

1.2 Either — erro como resultado, não como surpresa

O que é: em vez de uma função "explodir" com uma exceção quando algo dá errado, ela **retorna** um valor que já avisa se deu certo ou errado.

Analogia da caixa de correio com duas gavetas: imagine uma caixa de correio com duas gavetas etiquetadas — uma "Aprovado" e outra "Recusado, motivo: X". Quem vai buscar a carta **é obrigado a abrir uma gaveta específica** para saber o resultado; não existe um jeito de pegar a carta sem antes checar qual gaveta ela está. É diferente de um alarme que dispara aleatoriamente (a exceção "solta"): aqui, todo caminho de erro possível já está catalogado e visível antes mesmo de rodar o código.

Por que escolhemos: o compilador (TypeScript) **obriga** o programador a tratar o caso de erro antes de usar o resultado — não existe "esqueci de tratar aquele erro" silencioso, que é uma das causas mais comuns de bug em produção.

1.3 Casos de uso (use cases) — uma ação, uma responsabilidade

O que é: cada ação do sistema (aprovar um pedido, criar uma resposta, atualizar um perfil) é uma classe pequena com um único método `execute()`.

Analogia da linha de montagem: em vez de uma pessoa só fazendo o carro inteiro do início ao fim, cada estação da linha faz **uma coisa** e faz bem — uma estação instala o motor, outra a porta. Se um dia quisermos mudar como o motor é instalado, mexemos só naquela estação, sem parar a linha inteira nem reformar a estação da porta.

Por que escolhemos: fica fácil achar "onde mora" a regra de qualquer ação do sistema (é sempre um arquivo, com um nome que já diz o que faz), e fica fácil testar cada ação isoladamente.

1.4 Repositório como interface — a tomada elétrica universal

O que é: o caso de uso nunca fala diretamente com o banco de dados. Ele fala com uma "interface" (um contrato) — e uma peça separada, na camada de infraestrutura, é quem de fato conversa com o Postgres.

Analogia da tomada elétrica: qualquer aparelho eletrônico (a regra de negócio) é fabricado para encaixar num **padrão de tomada** — ele não sabe, nem precisa saber, se a energia vem de uma usina hidrelétrica, de um gerador a diesel ou de painel solar. Se a cidade trocar a fonte de energia, o aparelho continua funcionando do mesmo jeito, porque ele só conhece o formato do plugue, não a usina por trás.

Por que escolhemos: em teste automatizado, "plugamos" um gerador falso (um repositório de mentira, em memória) no lugar do banco de dados real — o teste roda em milissegundos, sem precisar subir Postgres. Em produção, plugamos o banco de verdade. O código da regra de negócio é idêntico nos dois casos.

1.5 Eventos de domínio — o aviso via alto-falante

O que é: quando algo importante acontece (uma resposta é criada, um pedido muda de status), o sistema "anuncia" esse fato. Outras partes do sistema que têm interesse "escutam" o anúncio e reagem — sem que a parte que anunciou precise saber quem está ouvindo.

Analogia do alto-falante do supermercado: quando o caixa anuncia "Cliente João, seu produto chegou no balcão" pelo alto-falante, ele não sabe (nem precisa saber) quem exatamente vai ouvir e agir — pode ser o segurança, pode ser um atendente. O caixa só faz o anúncio; quem tem interesse, reage.

Por que escolhemos: o módulo que cria a resposta não precisa importar nada do módulo de notificações — eles ficam desacoplados. O anúncio só é feito **depois** que a ação foi de fato salva no banco (não adianta anunciar "chegou" antes de conferir que chegou de verdade).

Ressalva honesta para a apresentação: esse "alto-falante" é interno ao processo, não uma fila de mensageria de verdade (tipo Correios registrado, com rastreamento e garantia de entrega). Se o anúncio se perder (o processo cair no meio), não há reenvio automático — aceitável para o volume atual, mas é um ponto de atenção se a criticidade da notificação aumentar.

1.6 Cache — o caderninho de anotações rápidas

O que é: dados que são lidos com muita frequência e mudam pouco ficam guardados por um tempo curto em um "banco de dados na memória" (Redis), evitando ir ao banco de dados principal toda vez.

Analogia do caderninho na mesa: em vez de ir até o arquivo morto no porão toda vez que alguém pergunta o mesmo dado, você anota num caderninho na sua mesa e consulta ali primeiro. Se o dado mudar, você rasura o caderninho (invalida o cache) para não responder algo desatualizado. E, de tempos em tempos, mesmo sem mudança nenhuma, você joga a anotação fora (TTL — tempo de expiração) para garantir que nunca fica velha demais.

Por que escolhemos: reduz a carga no banco principal e acelera a resposta para o usuário final, sem correr o risco de servir dado desatualizado por muito tempo.

1.7 Autenticação — o crachá de acesso

O que é: o usuário se autentica uma vez e recebe um token assinado digitalmente (JWT), que funciona como prova de identidade nas próximas requisições, sem precisar logar de novo a cada clique.

Analogia do crachá de visitante: ao chegar na recepção de um prédio, você mostra documento uma vez e recebe um crachá com um selo que **só a empresa sabe fabricar** (a assinatura digital). Em qualquer andar, o segurança olha o selo do crachá — não precisa ligar para a recepção de novo para confirmar quem você é. Se o crachá não tiver o selo correto, ou estiver vencido, o acesso é negado.

Por que escolhemos: evita que cada requisição precise validar login e senha de novo contra o banco de dados — o "selo" (assinatura) já garante a identidade.

Parte 2 — Frontend (o painel que o usuário vê)

2.1 React + Vite — o motor e a fábrica

O que é: React é a biblioteca que monta a tela em pedaços reutilizáveis (componentes); Vite é a ferramenta que empacota tudo isso para rodar rápido no navegador, e que recarrega a tela instantaneamente enquanto o time está codando.

Analogia do Lego e da esteira de montagem: React é como montar um painel com peças de Lego — cada peça (botão, tabela, card) é reutilizável em várias telas diferentes. Vite é a esteira que leva essas peças montadas até a vitrine (o navegador) o mais rápido possível, e sempre que uma peça é trocada, a esteira já entrega a atualização quase instantaneamente, sem ter que desmontar e remontar a vitrine inteira.

2.2 TanStack Query — o garçom com memória boa

O que é: biblioteca que busca dados da API, guarda em cache no navegador, e sabe quando precisa buscar de novo.

Analogia do garçom: um garçom experiente não vai à cozinha perguntar de novo "o prato X ainda está disponível?" toda vez que um cliente pergunta a mesma coisa dois minutos depois — ele lembra da última resposta (cache) por um tempo razoável. Mas se alguém faz um pedido novo na cozinha (uma mutação, tipo "cancelar pedido"), ele atualiza sua memória na hora e, se precisar, confirma com a cozinha de novo depois.

Por que escolhemos: sem essa biblioteca, o time teria que programar manualmente "quando buscar de novo", "onde guardar o que já foi buscado", "o que fazer se der erro" — para toda tela. A biblioteca já resolve isso de forma testada e madura.

2.3 Atualização otimista — o caixa de banco que confia primeiro

O que é: ao clicar em uma ação (ex.: "aprovar pedido"), a tela já muda imediatamente, **antes** da confirmação do servidor chegar. Se o servidor recusar, a tela volta atrás sozinha.

Analogia do caixa eletrônico: quando você deposita um cheque, o caixa eletrônico às vezes já mostra o saldo atualizado na hora, mesmo o cheque ainda não tendo sido 100% compensado pelo banco. Se o cheque voltar sem fundos depois, o saldo é ajustado de volta. A experiência do cliente é mais fluida — ele não fica esperando o processamento completo para ver algo acontecer na tela.

Por que escolhemos: a interface parece instantânea para quem está usando, mesmo que a rede tenha uma latência de alguns milissegundos ou segundos — sem esconder o fato de que, se algo falhar de verdade, o usuário é avisado e a tela é corrigida.

2.4 Design system (shadcn/ui) — peças de montar padronizadas

O que é: em vez de cada tela desenhar um botão, uma caixa de diálogo ou uma tabela do zero, existe um conjunto de peças prontas, acessíveis (funcionam bem com teclado e leitor de tela) e visualmente consistentes, que o time copia para dentro do próprio projeto (não é uma dependência fechada de terceiro).

Analogia da caixa de peças de Lego padronizadas de uma fábrica de móveis: todo móvel novo usa os mesmos parafusos, dobradiças e puxadores padrão — assim qualquer pessoa da equipe monta qualquer móvel novo sem aprender um sistema diferente a cada vez, e a aparência final fica sempre consistente entre os móveis, mesmo sendo montados por pessoas diferentes.

Por que escolhemos: consistência visual em todas as telas, acessibilidade "de fábrica" (não é preciso reimplementar suporte a teclado/leitor de tela em cada componente), e liberdade total para customizar, já que a peça mora dentro do nosso próprio projeto.

2.5 Tailwind CSS — etiquetas prontas em vez de costurar do zero

O que é: em vez de escrever CSS customizado para cada elemento, usamos combinações de classes utilitárias curtas e prontas (ex.: `flex`, `gap-4`, `text-sm`) diretamente no HTML/JSX.

Analogia das etiquetas de roupa prontas: é como ter um estoque de etiquetas de roupa já prontas — "manga curta", "azul", "tamanho M" — e simplesmente colar as combinações certas em cada peça, em vez de costurar uma etiqueta nova para cada roupa. Ganha-se velocidade e padronização; a "linguagem" das etiquetas é a mesma em toda a fábrica.

Por que escolhemos: acelera muito a velocidade de desenvolvimento de tela, e evita a bagunça comum de "cada desenvolvedor escreve CSS de um jeito" — todo mundo usa o mesmo vocabulário de classes.

2.6 Formulários com validação (Zod + React Hook Form) — o porteiro que confere antes

O que é: antes de qualquer dado do formulário ser enviado para o servidor, ele passa por uma checagem local (é um e-mail válido? o campo obrigatório foi preenchido?).

Analogia do porteiro de evento: o porteiro confere o ingresso **na entrada**, antes de a pessoa entrar no salão — é muito mais barato e rápido barrar um ingresso inválido na porta do que descobrir lá dentro que a pessoa não deveria estar ali. Da mesma forma, barramos dado inválido no formulário, antes de gastar uma viagem de rede até o servidor.

Por que escolhemos: feedback instantâneo para o usuário (sem esperar resposta do servidor para saber que errou o e-mail), e menos carga de requisições inválidas chegando na API.

Parte 3 — Perguntas que a plateia pode fazer (e como responder)

"Isso não é complexo demais para o tamanho do projeto?"

A complexidade não desaparece em nenhuma arquitetura — ou ela fica organizada em camadas claras, ou ela vira "espaguete" espalhado pelo código com o tempo. Estamos pagando um pouco de estrutura upfront para não pagar juros altos de manutenção depois.

"E se precisarmos ir rápido, sem seguir todo esse processo?"

A estrutura em camadas não deixa o desenvolvimento mais lento no dia a dia — ela deixa mais rápido a partir do segundo ou terceiro módulo, porque o padrão já está definido e testado. O ganho de velocidade aparece na manutenção, não seria visível olhando só a primeira tela pronta.

"Por que não usar uma ferramenta pronta (low-code) em vez de codar tudo?"

Ferramentas prontas resolvem bem casos genéricos; nosso ganho aqui é regra de negócio sob medida e controle total de performance e customização, sem ficar limitado ao que a ferramenta permite.

"Quem mantém isso se um desenvolvedor sair do time?"

Justamente por isso a arquitetura separa responsabilidades com nomes e lugares previsíveis — qualquer novo desenvolvedor sabe onde procurar "a regra de aprovar um pedido" sem precisar entender o sistema inteiro primeiro.

Parte 4 — Roteiro sugerido de apresentação (~20–25 min)

1. **Abertura (2 min)** — a frase central: "separamos o que o sistema faz de com o que ele é feito" (Parte 0).
2. **Backend em 4 metáforas (8 min)** — prédio (camadas) → caixa de correio (Either) → linha de montagem (use cases) → tomada elétrica (repositório). Pule eventos/cache/auth para perguntas,

a menos que sobre tempo.

3. **Frontend em 3 metáforas (6 min)** — garçom (React Query) → caixa eletrônico (update otimista) → peças de Lego (design system).
4. **Por que essas escolhas, não outras (3 min)** — testabilidade, troca de infraestrutura sem reescrever regra, consistência visual.
5. **Perguntas (5–6 min)** — usar a Parte 3 como base de respostas.

Dica de apresentação: não tente explicar as 13 analogias todas — escolha 4 ou 5 que mais conversam com quem vai assistir (se a plateia for mais de produto, foque em React Query/update otimista/design system; se for mais técnica, inclua Either e repositório).